Aprendizaje automático

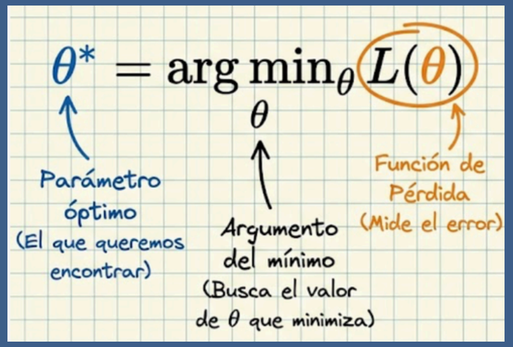
INCORPORACIÓN DE RESTRICCIONES EN MODELOS DE APRENDIZAJE AUTOMÁTICO

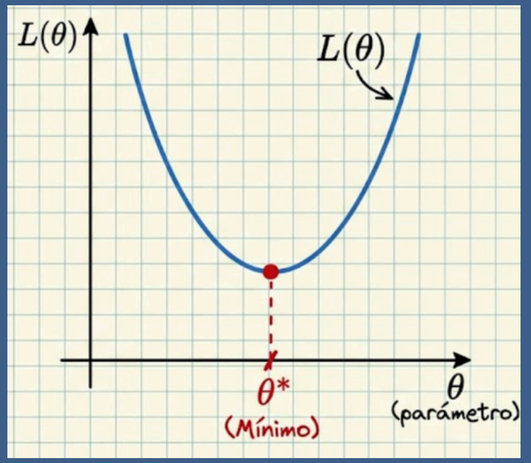
Manuela Rendón Sepulveda
Luciana Núñez Ospina
Cristian Miguel Gómez Salazar
Santiago Alberto Vélez Casallas

¿Qué son?

Las restricciones son condiciones, reglas o límites que se incorporan a un modelo de aprendizaje automático para orientar su comportamiento y asegurar que sus resultados cumplen determinados criterios.

Consiste en añadir reglas físicas, lógicas, éticas o de negocio directamente en el proceso de entrenamiento o en la función de pérdida para guiar las predicciones del modelo.

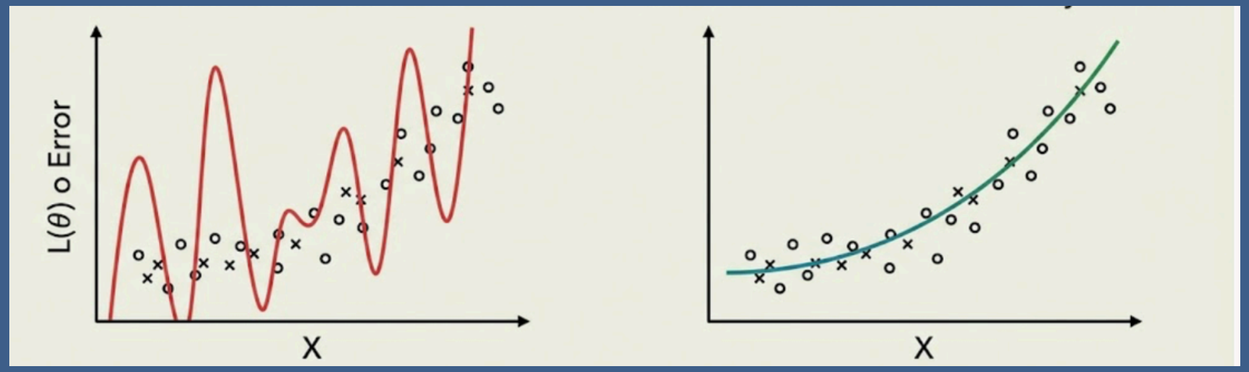




¿Por qué?

Permiten desarrollar modelos mucho más confiables, seguros y coherentes en el contexto en el que se utilizan. Ayuda a reducir estos problemas, orientar el aprendizaje hacia soluciones válidas y mejorar la utilidad de las predicciones para la toma de decisiones.

Sin embargo, su efectividad depende de que las restricciones estén bien definidas y sean compatibles con el problema.



En que etapa se incorporan las restricciones

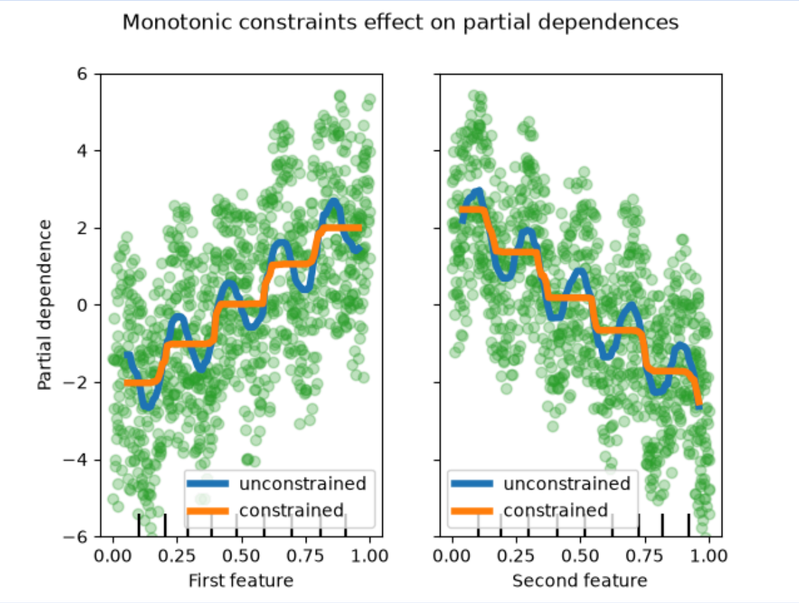
Entrenamiento →	Mediante funciones de pérdida con personalizaciones o condiciones sobre los parámetros
Inferencia →	Cuando el modelo genera predicciones, limitando las salidas posibles o guiando la búsqueda de soluciones válidas
Postprocesamiento →	Se ajustan las predicciones después de generarlas para que cumplan las condiciones establecidas

Principales tipos de restricciones en ML

Restricciones de monotonía

Indica que cuando cambia una variable específica y las demás permaneces constantes, la predicción debe moverse en una dirección determinada

- Monotonía creciente
- Monotonía decreciente
- Sin restricción



https://scikit-learn.org/stable/auto_examples/ensemble/plot_monotonic_constraints.html

Restricciones de rango o límites

Establecen los valores mínimos y máximos permitidos para una variable o predicción del modelo.

$$L \leq \hat{y} \leq U$$

- Límite duro: la salida siempre debe permanecer dentro del rango.
- Límite blando: el modelo puede incumplir el rango, pero recibe una penalización.

Se pueden aplicar a las entradas, salidas o parámetros del modelo.

- El rango determina cuáles predicciones son admisibles, pero no garantiza que sean precisas.
- Los límites deben provenir del conocimiento del dominio, no solamente de los valores mínimos y máximos observados en los datos.

Restricciones de conservación

Obligan al modelo a mantener constante una magnitud física conservada o a respetar su balance de entradas, salidas, generación y acumulación.

$$\text{Acumulación} = \text{Entradas} - \text{Salidas} + \text{Generación} - \text{Consumo}$$

- La cantidades se balancean
- No aparezcan recursos inexistentes
- No desaparezcan recursos sin explicación

Restricciones mediante ecuaciones físicas

Las predicciones del modelo satisfacen una ecuación conocida que describe el comportamiento del sistema.

Una ley de conservación puede ser una ecuación física, pero no toda ecuación física es una ley de conservación.

Restricciones de invariancia

Una restricción de invariancia establece que ciertas transformaciones de la entrada no deben modificar la predicción del modelo.

$$f(T(x)) = f(x)$$

Si la entrada cambia la salida debe permanecer igual

Restricciones de equivariancia

Las restricciones de equivariancia obligan al modelo a reflejar en la salida las transformaciones aplicadas a la entrada. Si la entrada cambia, la predicción también debe cambiar, pero de una manera coherente y predecible..

Regularización

La regularización es una técnica que añade una penalización al entrenamiento para evitar que el modelo sea demasiado complejo.

¿Por qué se utiliza?

Sin regularización:

- Ajustarse excesivamente a los datos de entrenamiento (overfitting).
- Memorizar ruido.
- Generar predicciones inestables.

Con regularización:

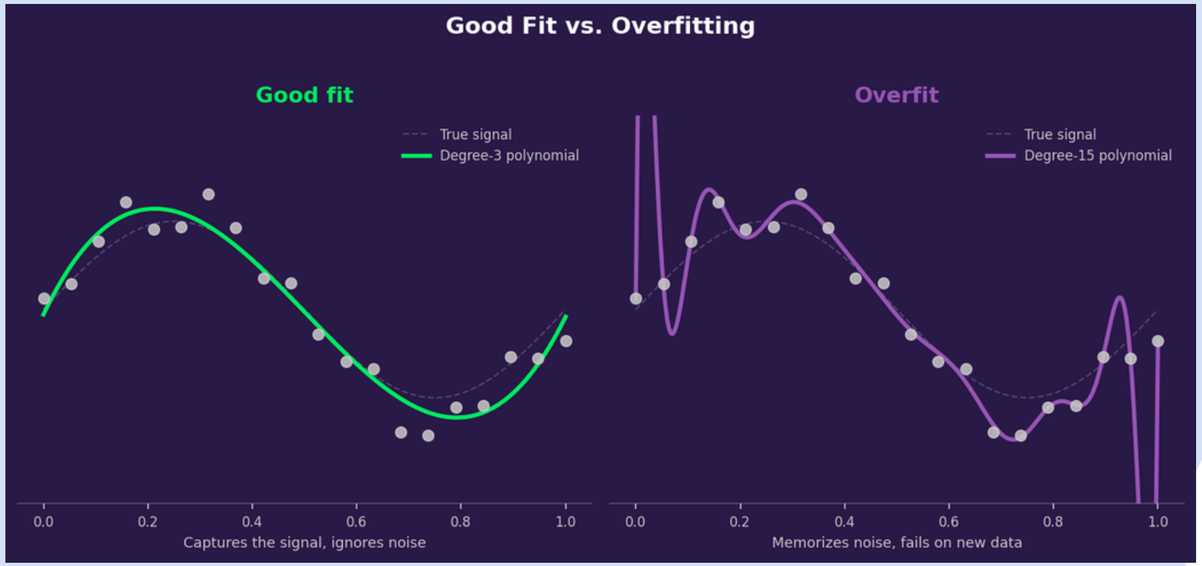
- Se favorecen modelos más simples.
- Mejora la capacidad de generalizar a datos nuevos.
- Reduce el riesgo de sobreajuste.

Se añade un costo

El modelo busca

Minimizar el error

Evitar soluciones excesivamente complejas que podrían no funcionar bien con datos nuevos.



https://www.datacamp.com/es/tutorial/regularization-in-machine-learning

Regularización

La regularización es una técnica que añade una penalización al entrenamiento para evitar que el modelo sea demasiado complejo.

En lugar de minimizar solo el error, ahora el modelo minimiza esto:

$$\text{Loss} + \lambda \cdot \text{Penalty}$$

L2 (Ridge)

$$\text{Loss} + \lambda \sum_j w_j^2$$

Reducir coeficientes demasiado grandes y evitar que el modelo dependa excesivamente de una sola variable

¿Cuándo usar Ridge?

- **Cuando crees que la mayoría de variables son útiles.**
- **Cuando existen variables correlacionadas.**
- **Cuando buscas estabilidad.**

L1 (Lasso)

$$\text{Loss} + \lambda \sum_j |w_j|$$

Añade una penalización proporcional al valor absoluto de los coeficientes. Puede reducir algunos coeficientes exactamente a cero, eliminando variables del modelo.

¿Cuándo usar Lasso?

- **Cuando tienes muchas variables.**
- **Cuando sospechas que varias son irrelevantes.**

Elastic Net

$$\text{Loss} + \lambda_1 \sum_j |w_j| + \lambda_2 \sum_j w_j^2$$

Lasso (L1) elimina variables poco relevantes, **Ridge (L2)** conserva todas las variables reduciendo su influencia, y ***Elastic Net*** combina ambos enfoques para obtener modelos más simples y estables.

¿CÓMO SE VE ESTO EN MODELOS REALES?

Monotonía en Ensambles de Árboles (XGBoost y LightGBM)

EL PROBLEMA: En datos tabulares con ruido, los árboles de decisión libres crean ramas contra-intuitivas

XGBoost

```python
import xgboost as xgb
# +1: creciente, -1: decreciente, 0: libre
params = {'monotone_constraints': (1, -1, 0)}
model = xgb.XGBRegressor(**params)
model.fit(X_train, y_train)
```

```python
import lightgbm as lgb

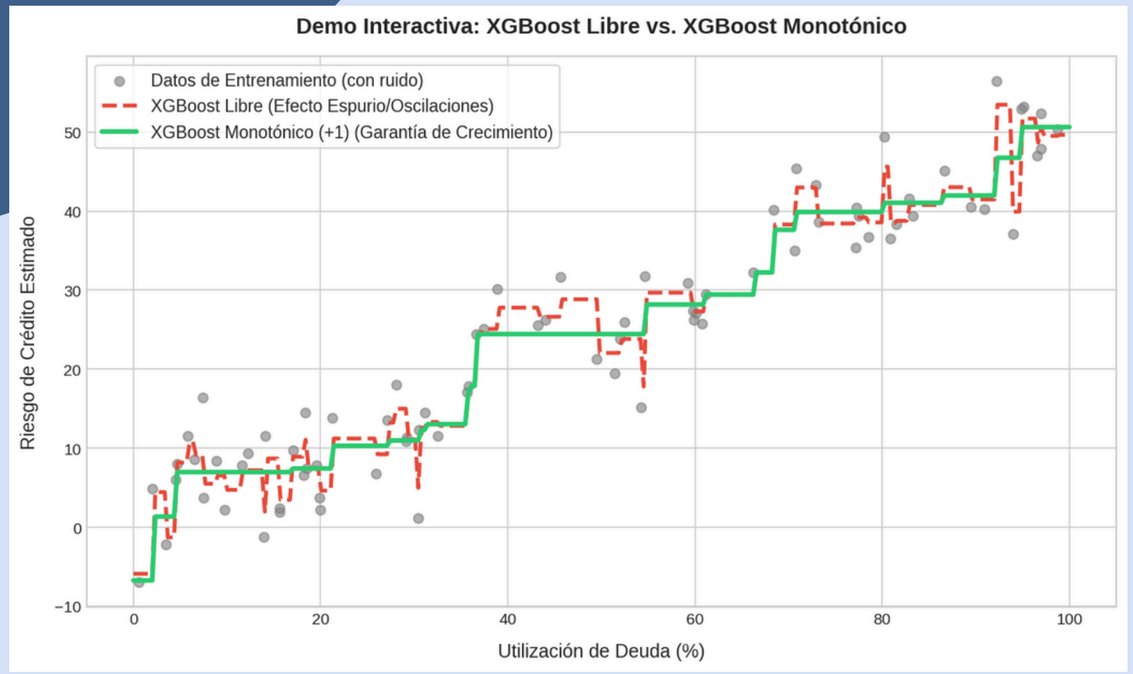
# Supongamos que las variables son: [potencia, vibracion, temperatura_ambiente]
# +1: Siempre creciente
# -1: Siempre decreciente
#  0: Libre (sin restricción)
params = {
    'monotone_constraints': [1, 1, 0]
}

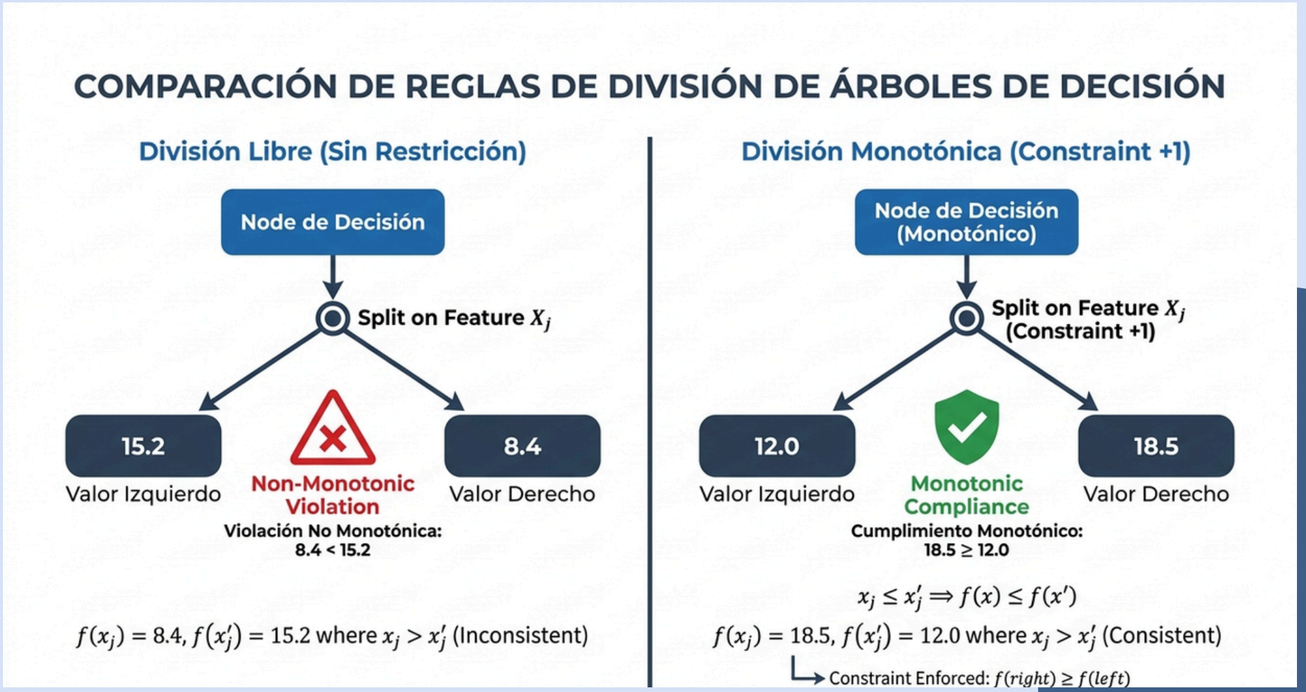
# Al entrenar, el algoritmo penaliza a infinito cualquier división
# de hojas que haga bajar la predicción cuando la potencia sube
train_data = lgb.Dataset(X_train, label=y_train)
model = lgb.train(params, train_data)
```

LightGBM

La solución matemática: restringir el criterio de partición (split) para que la función sea no decreciente . $(\partial f / \partial x \geq 0)$.

Restricciones de Monotonía en Árboles de Decisión y XGBoost / LightGBM





Restricciones en Redes Neuronales por Reparametrización y Arquitectura (Hard Constraints)

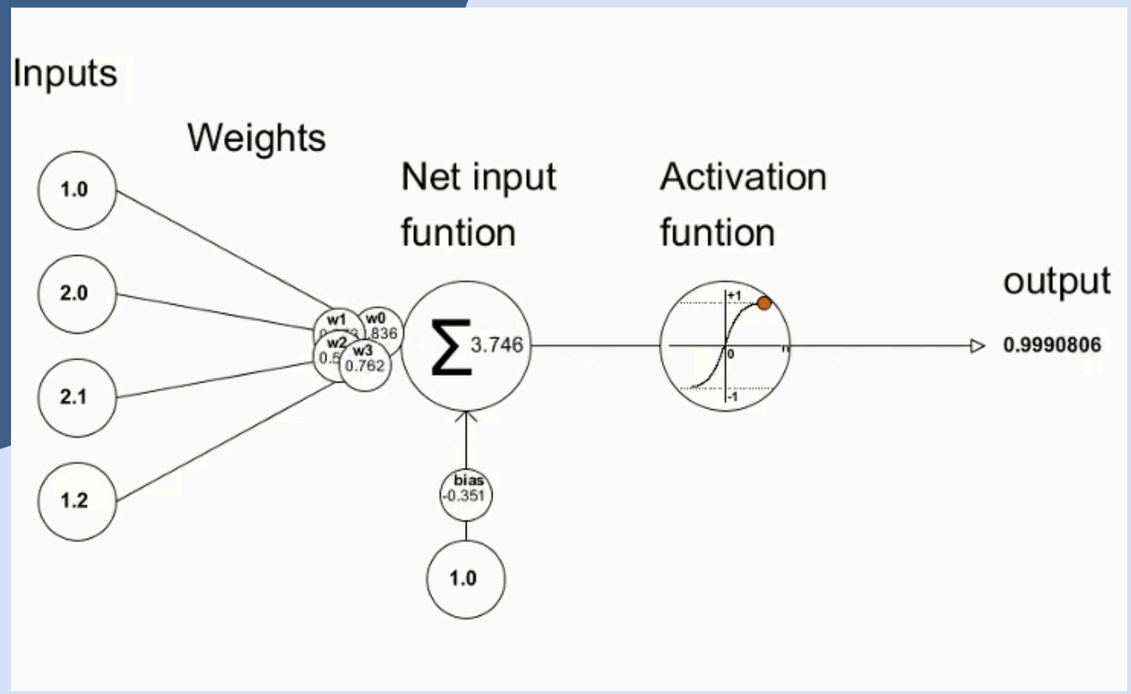
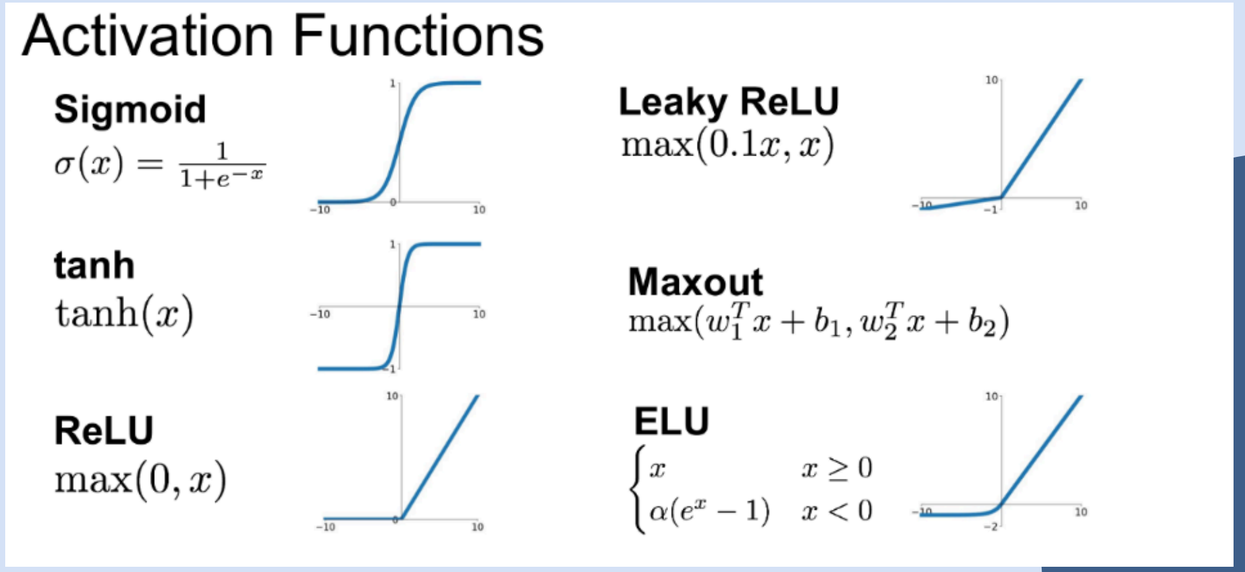
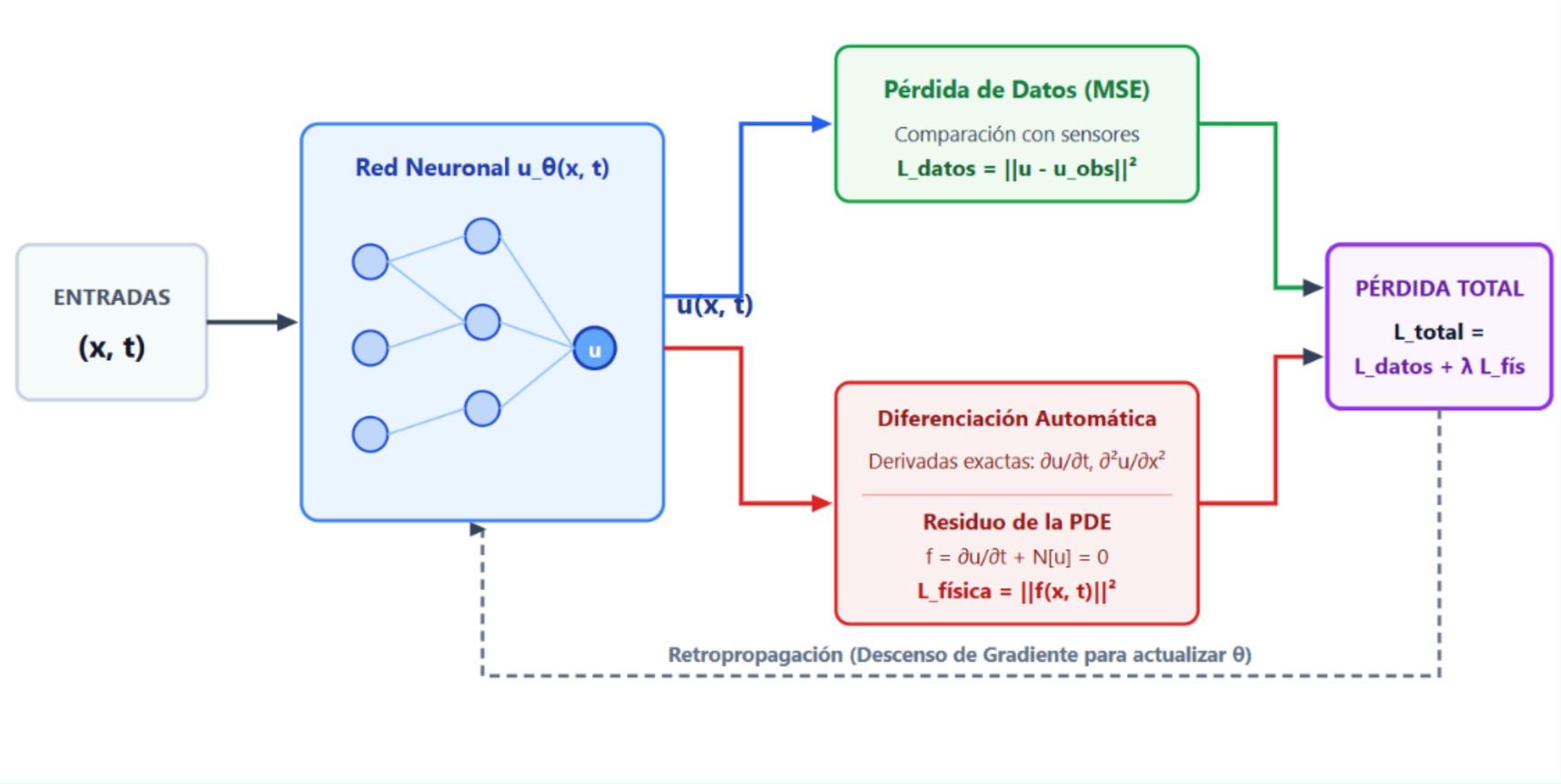


Diagrama de red neuronal que representa la capa de pesos

Funciones de activación monótonas crecientes



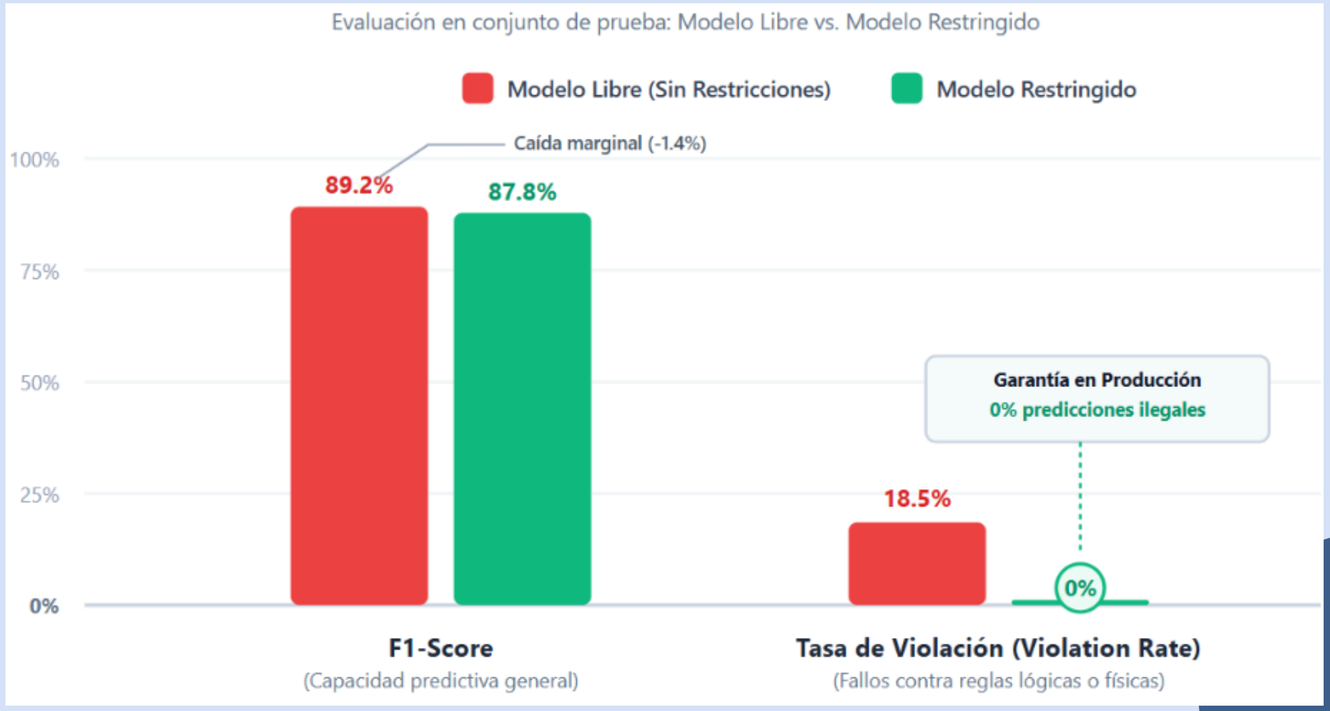
Physics-Informed Neural Networks (PINNs)



Evaluación: Ajuste y Validez

Violation Rate (Tasa de Violación): Porcentaje de casos de prueba donde el modelo contradice la regla lógica o física (en el modelo restringido debe ser 0%).

Límites Bounding: garantía de que las predicciones nunca caigan fuera de rangos admisibles.



Apliquémosle restricciones a un Transformer

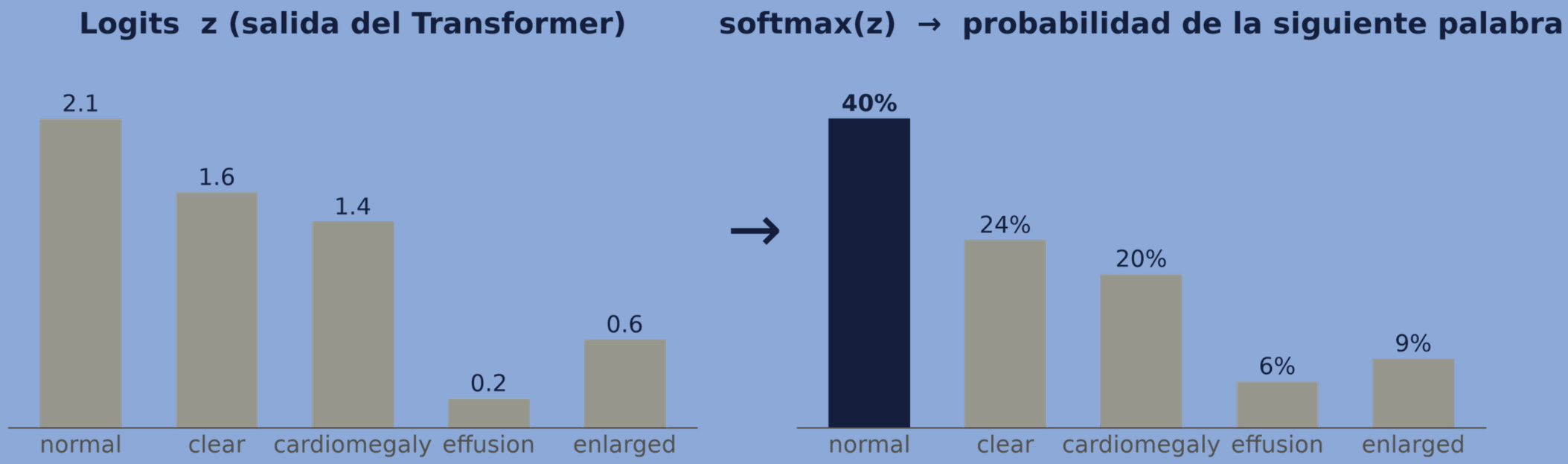
¿Qué restricciones clásicas sirven en un Transformer?

Modelo	Restricción típica	¿Sirve en un Transformer?
Regresión lineal / logística	Ridge / Lasso, signo de los coeficientes	✓ **L2 (weight decay)** ✗ **signos** ✓ **mover el logit**
Árbol de decisión	Profundidad máxima, hojas mínimas, poda	✗ **no hay cortes** (análogo: número de capas)
Random Forest	Bagging: promediar muchos árboles	✓ **promediar** (simetrías, varios razonamientos)
XGBoost / LightGBM	Monotonía, restricciones de interacción	✗ **se imponen al elegir cada corte**
Transformer	Datos, arquitectura, pérdida	✓ **y además: corregir los logits en la inferencia ← hoy**

Cómo escribe un Transformer: palabra a palabra

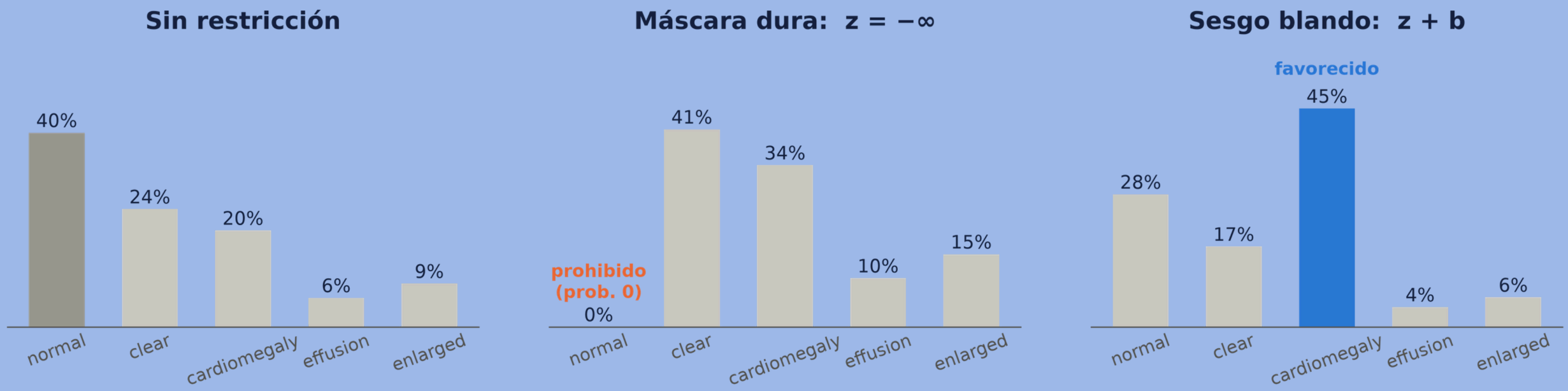
$$P(y_1 \dots yT \mid \text{imagen}) = \prod_t P(\, y_t \mid y_{(<t)}, \text{imagen}\,)$$

logits → softmax → probabilidad → palabra elegida → se agrega al texto y se repite



Todo pasa por el softmax: ahí es donde vamos a intervenir, sin tocar los pesos.

Restringir en los logits: duro o blando



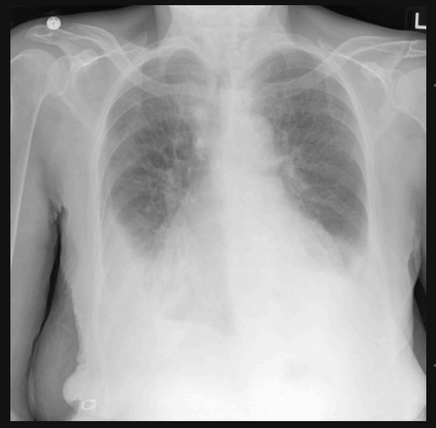
Dura: $z' = -\infty$

Probabilidad 0: la regla se cumple siempre.
PICARD (SQL válido), JSON con gramática

Blanda: $z' = z + b$

La regla se favorece; el modelo puede contradecirla.
CCD: la que usamos hoy

CCD: un experto corrige al Transformer palabra a palabra



Radiografía

DenseNet experto solo en 14 tipos de diagnóstico

p(cardiomegalia) = 0,82

b = clip(log p⁄(1−p) , ± log γ)

MAIRA-2 · 7B pesos congelados

logits z

z + β·b

softmax

“cardio-megaly”

Se corre 2 veces: con la instrucción normal y con una nota del experto (se mezclan con α)

$$P_{final}(v) \propto P_{orig}(v)^{1-\alpha} \cdot P_{clín}(v)^{\alpha} \cdot \left(\frac{p}{1-p}\right)^{\beta}$$

α = β = 0,5 · γ = 10 → el experto multiplica o divide una palabra como máximo por $\sqrt{10} \approx 3{,}16$

Ningún peso del Transformer cambia

Zhang, Meng, Lever & Ho. CCD: Clinical Contrastive Decoding. ACL 2026.

0,03 → 0,15

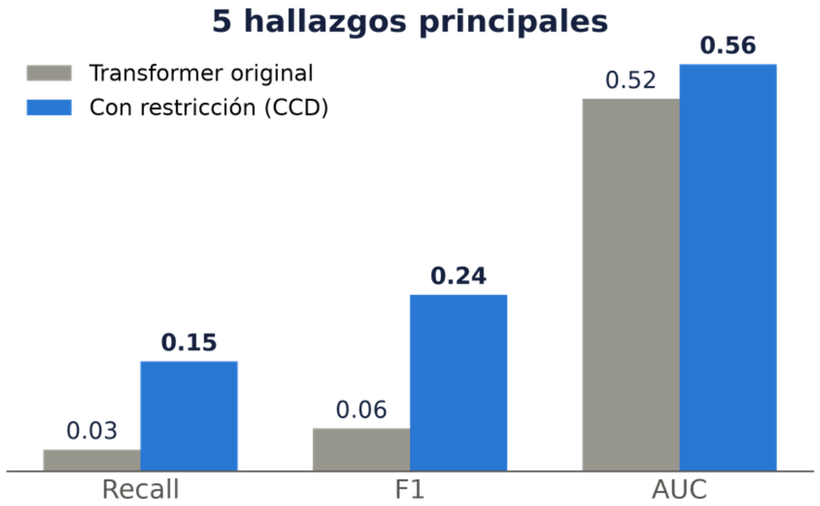
Recall · 5 hallazgos principales (×5)

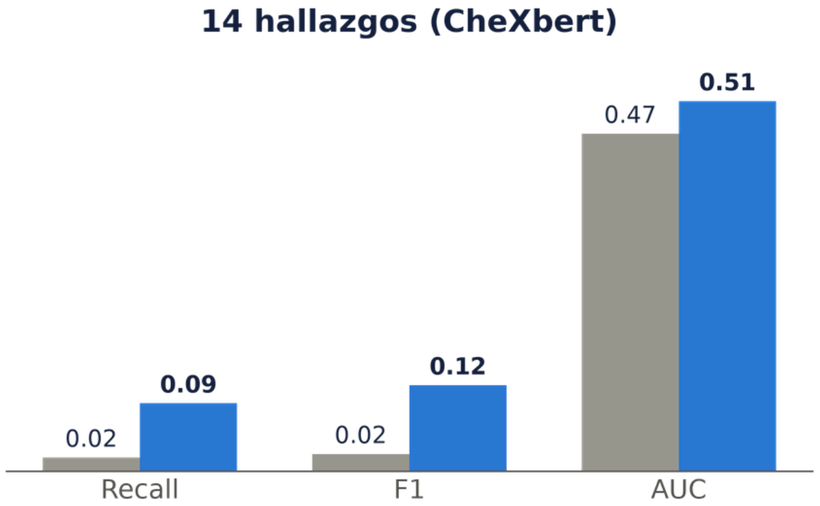
0,06 → 0,24

F1 · 5 hallazgos principales (×4)

95–98 %

P(mejora) · 14 hallazgos (bootstrap)

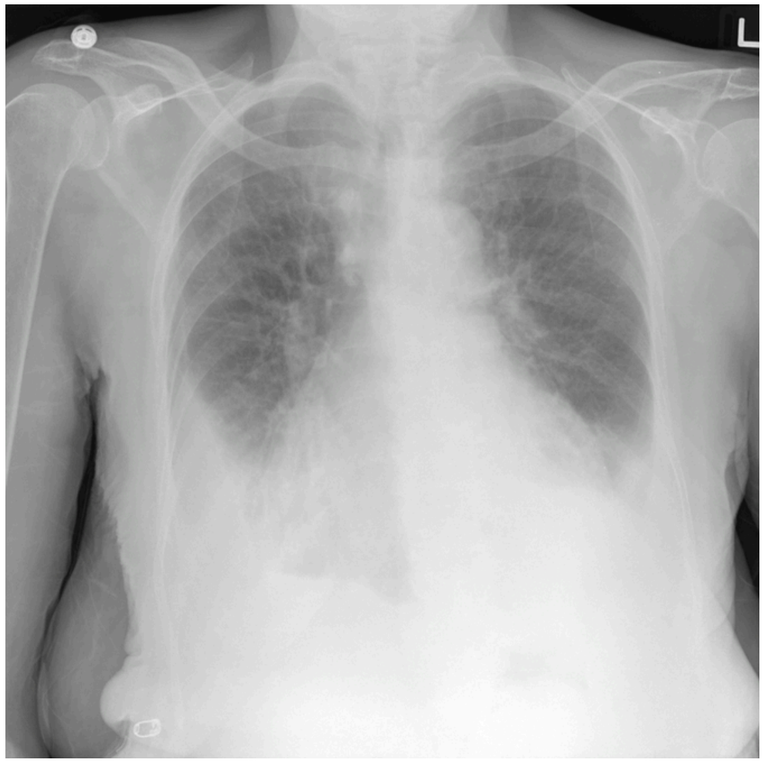




n = 30 estudios con hallazgos
IU X-Ray · evaluación con CheXbert ·

Paper (3307 estudios):
F1 4,02 → 19,96

Repositorio: github.com/santiagovelcasallas/incorporacion-restricciones-aprendizaje-automatico



Experto: p(cardiomegalia) = 0,82

Radiólogo: corazón aumentado de tamaño (cardiomegalia) y opacidades intersticiales

Transformer original ✗ omite la cardiomegalia

«The lungs are well expanded and clear… Cardiac and mediastinal contours are normal.»

Con restricción (CCD) ✓ detecta la cardiomegalia

«The cardiac silhouette is mildly enlarged. The mediastinal and hilar contours are within normal limits.»

Sigue sin mencionar las opacidades: la restricción ayuda, no reemplaza al radiólogo.

IU X-Ray (Open-i), estudio 148 del fragmento evaluado.

Ventajas

1

zero reentrenamiento del trasnformer

Los pesos de MAIRA-2 (7 000 millones de parámetros) no se tocan.

2

Modular e intercambiable

El experto se cambia sin tocar el Transformer: el código oficial ya trae 4 (DenseNet, MedSigLIP, uno de vista y uno de dispositivos). Si mañana sale un experto mejor, se enchufa y listo.

3

Controlable y medible

su efecto se mide con métricas estándar: recall de 0,03 → 0,15. y β y γ son ajustables

Alcance y desventajas

1

Blanda ≠ garantía

El Transformer puede contradecir al experto.

2

Hereda al experto

Si el DenseNet se equivoca, empuja mal. Aquí su AUC es 0,61.

3

Costo en inferencia

De 1,4 s a 11,5 s por informe: el modelo se corre dos veces por palabra.

4

Solo ayuda si el modelo viola la regla

BiomedCLIP ya era invariante al giro: +0,007 de AUC.

Texto guiado ≠ diagnóstico correcto

Igual que “estructura válida ≠ verdad”: la restricción vale lo que vale el conocimiento que la alimenta.

Aprender de los datos + incorporar conocimiento previo = reducir el espacio de soluciones admisibles

github.com/santiagovelcasallas/incorporacion-restricciones-aprendizaje-automatico

¡Gracias!